

Design Patterns în C#

Un pattern nu e o bibliotecă și nu e sintaxă nouă — e un nume pentru o soluție care se repetă. Fiecare lecție pornește de la codul naiv, arată durerea, apoi forma care o rezolvă — și spune și când *nu* merită folosită.

[Strategy](#)[Observer](#)[Factory Method](#)[Decorator](#)

Lecția 0 — Ce este un design pattern (puntea de la interfețe)

De unde venim

La lecția trecută ai învățat **interfețele**: o clasă semnează un contract (`can-do`), iar restul codului depinde de contract, nu de clasa concretă. `Desen` lucra cu `IElement[]` fără să-i pese CE sunt elementele — doar CĂ respectă contractul.

Ține minte propoziția asta, pentru că e singura idee din tot modulul:

Depinzi de contract, nu de clasă.

Toate cele 8 pattern-uri pe care le facem sunt aceeași idee, aplicată la trei întrebări diferite.

Ce e, de fapt, un pattern

Un **design pattern** nu e o bibliotecă, o clasă gata făcută sau o sintaxă nouă. E un **nume pentru o soluție care se repetă** la o problemă care se repetă. Cineva a lovit aceeași problemă de o mie de ori, a găsit forma care merge, și i-a dat un nume ca să putem vorbi despre ea în două cuvinte în loc de zece minute.

Beneficiul e dublu:

- **Nu reinventezi** — când recunoști problema, știi deja forma soluției.
- **Comunici scurt** — spui „aici e un Strategy” și tot echipa înțelege structura, fără să citească fiecare linie.

Riscul, pe care îl vom evita: pattern-ul aplicat unde nu trebuie. Un pattern rezolvă o durere concretă. **Fără durere, fără pattern** — altfel adaugi complexitate degeaba.

Firul logic al modulului — trei întrebări

Grup	Întrebarea la care răspunde	Pattern-uri
Comportament	<i>cine se comportă și cum?</i>	Strategy, Observer, State
Structură	<i>cum compun obiectele?</i>	Decorator, Adapter
Creare	<i>cine și cum creează obiectele?</i>	Factory Method, Builder, Singleton

Ordinea nu e întâmplătoare: începem cu **comportamentul**, pentru că e cel mai aproape de interfețe. Primul pattern — Strategy — e practic un pas mic peste ce știi deja.

Lecția 1 — Strategy

Un cuvânt: comportament interschimbabil. Închizi fiecare variantă de „cum se face ceva” într-o clasă separată care semnează același contract, și o schimbi când vrei — chiar la runtime.

Problema (exercițiul `ex1` : costul de livrare)

Un magazin calculează costul de livrare al unei comenzi. Costul depinde de metodă: standard, express, promoție gratuită. Prima variantă la care se gândește oricine:

```
public decimal CalculeazaCost(TipLivrare tip, decimal greutate, decimal distanta)
{
    switch (tip)
    {
        case TipLivrare.Standard: return 5 + 0.5m * greutate + 0.1m * distanta;
        case TipLivrare.Express:  return 12 + 1.0m * greutate + 0.25m * distanta;
        case TipLivrare.Gratuita:  return 0;
        default: throw new ArgumentException("Unknown delivery type");
    }
}
```

Merge. Și e o capcană. Întreabă-te ce faci când firma adaugă `LivrareInternationala` :

- deschizi metoda asta și adaugi un `case` ;
- probabil mai există un `switch` la fel în alt loc (afișare, facturare) — îl deschizi și pe ăla;
- fiecare `switch` uitat = un bug tăcut.

Ai lovit exact regula **open/closed** de la interfaces ex5: o clasă ar trebui să fie deschisă la extindere, dar închisă la modificare. Aici, ca să extinzi (metodă nouă), ești **obligat să modifice** cod care deja mergea. `switch` -ul pe tip e mirosul clasic care strigă „aici lipsește un pattern”.

Ideea

Întoarce lucrurile pe dos. În loc ca o metodă să știe TOATE variantele și să aleagă cu `switch` , fiecare variantă devine o clasă care știe DOAR de ea și semnează un contract comun:

```
public interface ILivrareStrategie
{
    string Nume { get; }
    decimal CalculeazaCost(decimal greutateKg, decimal distantaKm);
}
```

Comanda nu mai conține nicio formulă și niciun **switch**. Ține o **ILivrareStrategie** și o **întreabă**:

```
public decimal CostTransport()
{
    return strategie.CalculeazaCost(GreutateKg, DistanțaKm);
}
```

O metodă nouă de livrare = **o clasă nouă**, atât. Nu atingi **Comanda**, nu atingi celelalte strategii. Închis la modificare, deschis la extindere — de data asta chiar.

Cele trei roluri

Rol	În exercițiu	Ce face
Strategy (contractul)	ILivrareStrategie	declară operația interschimbabilă
ConcreteStrategy	LivrareStandard , LivrareExpress , LivrareGratuita	fiecare, o singură variantă a algoritmului
Context	Comanda	ține o strategie, deleagă spre ea, o poate schimba



Mecanismul de dedesubt

Nu e magie — sunt trei lucruri pe care le știi deja, puse împreună:

1. **Polimorfism prin contract.** `strategie.CalculeazaCost(...)` cheamă implementarea clasei reale din spatele interfeței. Exact ce făcea `IElement.Afisare()` la desen.
2. **Delegare.** `Comanda` nu face calculul, îl pasează mai departe. Contextul e un dispecer, nu un executant.
3. **Injectare.** Strategia intră din afară (prin constructor sau `SchimbaStrategie`), nu e creată înăuntru. De-aia o poți schimba la runtime — și de-aia contextul nu depinde de nicio clasă concretă.

Detaliul care le leagă: `CalculeazaCost` primește greutatea și distanța ca **parametri**, nu le ține strategia în câmpuri. Așa aceeași instanță `LivrareStandard` servește orice comandă — strategia e *fără stare*, deci refolosibilă și fără surprize. (Vezi întrebarea din bonusul exercițiului.)

Când îl folosești

- Ai **mai multe variante** ale aceluiași lucru (calcul, sortare, validare, format de export) și vrei să comuți între ele.
- Vezi un `switch /lanț de if` pe un „tip” care se tot repetă prin cod.
- Vrei să **adaugi variante** fără să atingi codul existent.
- Vrei să poți schimba comportamentul **la runtime**, nu doar la compilare.

Când NU îl folosești

- Ai **o singură variantă** și nu se întrevăd altele. Un `if` simplu e mai onest decât trei clase și o interfață.

- Variantele diferă printr-o **singură valoare**, nu printr-un algoritm (atunci e un parametru, nu o strategie).
- Pattern-ul aplicat de dragul pattern-ului adaugă doar ceremonie. Amintește-ți: fără durere, fără pattern.

Capcane frecvente

- **Context cu switch rămas.** Dacă `Comanda` tot decide cu `if` care strategie să folosească, n-ai mutat problema — ai dublat-o. De-aia constrângerea din exercițiu interzice `if` / `switch` pe tip în `Comanda`.
- **Strategie cu stare de comandă.** Dacă `LivrareStandard` ține greutatea în câmp, n-o mai poți refolosi între comenzi. Ține strategia fără stare; datele vin ca parametri.
- **Prea multe strategii minuscule.** Dacă variantele diferă cu o linie, poate era destul un `Func<>` sau un parametru.

Legătura cu ce urmează

În bonusul exercițiului ai scris `LivrareCuReducere`: o strategie care primește **altă strategie** în constructor și îi ajustează rezultatul. O strategie care împachetează o strategie — asta e deja un **Decorator** (lecția 4). Strategy ți l-a arătat gratis; la lecția următoare doar îi punem numele.

Lecția 2 — Observer

Un cuvânt: un eveniment, N reacții. O sursă își schimbă starea și îi anunță pe toți cei abonați — fără să știe cine sunt sau ce fac.

Problema (exercițiul **ex2** : starea comenzii)

Aceeași comandă, dar acum urmărim starea ei: **Plasata → Expediata → Livrata**. La fiecare schimbare, mai multe părți reacționează: clientul primește email, se scrie în jurnal, depozitul se pregătește. Prima variantă:

```
public void SchimbaStare(string stareNoua)
{
    Stare = stareNoua;
    email.Trimite(stareNoua);
    jurnal.Scrie(stareNoua);
    depozit.Pregateste(stareNoua);
}
```

Comanda ajunge să cunoască fiecare parte pe nume. Marketingul cere o notificare push → redeschizi **SchimbaStare** și mai adaugi o linie. Iar o clasă care ar trebui să se ocupe de *comandă* știe acum despre email, SMS, jurnal, depozit, push. Aceeași durere de open/closed ca la Strategy — dar pe altă axă: la Strategy se schimbă **cum se calculează**, aici se schimbă **cine reacționează**.

Ideea

Inversezi dependența. **Comanda** (rolul de **Subject**) nu mai cunoaște nicio parte concretă — ține o listă de observatori care au semnat un contract, și când se schimbă starea îi anunță pe toți, la fel:

```

public interface IObservable
{
    void Actualizeaza(string stareNoua);
}

public void SchimbaStare(string stareNoua)
{
    Stare = stareNoua;
    for (int i = 0; i < observatori.Length; i++)
    {
        observatori[i].Actualizeaza(stareNoua);
    }
}

```

O parte nouă care vrea să reacționeze = o clasă nouă care implementează `IObservable` . `Comanda` rămâne neatinsă.

Cele trei roluri

Rol	În exercițiu	Ce face
Subject (sursa)	<code>Comanda</code>	ține observatorii, își schimbă starea, îi anunță pe toți
Observer (contractul)	<code>IObservable</code>	declară <code>Actualizeaza(...)</code>
ConcreteObserver	<code>NotificatorEmail</code> , <code>JurnalLivrare</code> , <code>PanouDepozit</code>	fiecare reacționează în felul lui

```

Comanda (Subject)
- observatori: IObservable[]
- SchimbaStare() -> anunta toti
    |
+-----+-----+
v         v         v
NotificatorEmail JurnalLivrare PanouDepozit
(fiecare semneaza IObservable)

```

Strategy vs Observer — aceeași unealtă, altă direcție

Amândouă folosesc un contract și polimorfism. Diferența e **direcția** și **numărul**:

	Strategy (ex1)	Observer (ex2)
Câți colaboratori	UNU (strategia curentă)	MULȚI (toți observatorii)
Ce face contextul	îi cere ceva: „cât costă?”	îi anunță: „s-a schimbat starea”
Așteaptă răspuns?	DA — primește un cost înapoi	NU — anunță și merge mai departe
Cine e în control	contextul întreabă când vrea	evenimentul împinge spre toți

„A întreba” vs „a anunța” — asta e distincția de fixat. Observer nu așteaptă nimic înapoi; de-aia `Actualizeaza` returnează `void`.

Când îl folosești

- O schimbare într-un loc trebuie să declanșeze reacții în **mai multe** locuri independente.
- Nu vrei ca sursa să cunoască reacțiile (le poți adăuga/scoate fără s-o atingi).
- Reacțiile nu trebuie coordonate între ele și nu întorc nimic sursei.

Când NU îl folosești

- Ai **un singur** ascultător și va rămâne unul — un apel direct e mai clar.
- Ai nevoie de un **răspuns** de la fiecare parte (atunci nu e Observer — e altceva).
- Ordinea reacțiilor contează strict sau una depinde de alta — Observer nu garantează asta.

Capcane frecvente

- **Subject care cunoaște observatorii concreți.** Dacă în `SchimbaStare` apare `if (o is NotificatorEmail)`, ai ratat pattern-ul — sursa trebuie oarbă la tipuri. De-aia constrângerea interzice `is / as`.
- **Observer care aruncă și oprește lanțul.** Dacă al doilea observator crapă, al treilea nu mai e anunțat. (La proiecte reale se izolează fiecare; la noi, ține-i simpli.)
- **Notificare fără schimbare reală.** Anunță doar când starea chiar s-a schimbat, altfel observatorii reacționează degeaba.

Legătura cu ce urmează

`NotificatorEmail`, `JurnalLivrare` reacționează fiecare *în plus* la un eveniment. Când vei vrea nu părți separate care reacționează, ci să **adaugi comportament peste un obiect existent**, împachetându-l — acolo intră **Decorator** (grupul *Structură*). Deocamdată reține doar contrastul: Observer pune reacții *alături*; Decorator le pune *în jurul*.

Lecția 3 — Factory Method

Un cuvânt: creare delegată. Muți `new` din codul care decide, în clase care știu fiecare să nască un singur lucru.

Notă de parcurs. În firul anunțat la lecția 0, aici urma State, iar Factory Method venea în grupul *Creare*. Îl luăm înaintea pentru că ai lovit deja problema pe care o rezolvă, singur, în proiectul tău *academy*. Un pattern se învață cel mai bine când doare — iar asta te doare acum.

Problema (exercițiul `ex10` : încărcarea utilizatorilor)

Ai un fișier în care fiecare linie începe cu un tip:

```
STUDENT,Ana,Popescu,anul 2
PROFESOR,Ion,Ionescu,Matematica
STUDENT,Vlad,Marin,anul 1
```

Trebuie să construiești, pentru fiecare linie, obiectul potrivit: `Student` sau `Profesor`. Prima variantă la care se gândește oricine:

```
foreach (string linie in linii)
{
    string[] campuri = linie.Split(',');
    switch (campuri[0])
    {
        case "STUDENT": utilizatori.Add(new Student(campuri)); break;
        case "PROFESOR": utilizatori.Add(new Profesor(campuri)); break;
        default: throw new ArgumentException("Unknown user type");
    }
}
```

Merge. Și e exact `switch`-ul de la lecția 1 — aceeași durere, alt loc: ca să adaugi `ADMIN`, redeschizi o metodă care mergea.

Dar de data asta Strategy **nu** te salvează, și merită înțeleles de ce.

De ce polimorfismul NU rezolvă asta

În `academy` ai încercat exact asta, și e o încercare bună — instinctul era corect. Ai scos `switch` -ul și ai pus polimorfism:

```
User newUser = new();
newUser.Create(request);
```

`Create` e `virtual`, `Teacher` și `Admin` îl suprascriu. Deci ar trebui să meargă, nu?

Nu. Uită-te la prima linie și întreabă-te **ce tip are obiectul**.

E un `User`. L-ai scris tu, acolo: `new User()`. Apelul virtual se duce la implementarea **tipului real al obiectului** — iar tipul real e `User`. Deci se cheamă `User.Create`, niciodată `Teacher.Create`. Overrides-urile tale nu se execută nici măcar o dată.

Aici e propoziția de reținut din toată lecția:

Polimorfismul alege ce METODĂ rulează pe un obiect care există deja. Nu poate alege ce CLASĂ se construiește.

Când ajungi la `new`, decizia e deja luată — ai scris tu numele clasei, în cod, la compilare. Polimorfismul intră în scenă *după*. Nu ajunge niciodată destul de devreme.

„Cine decide ce clasă se naște” e o **altă întrebare** decât „cine decide cum se comportă”. Prima are nevoie de alt pattern.

Ideea

Dacă `new Student(...)` nu poate fi ales polimorfic, atunci ascunde-l în spatele a ceva care **poate** fi ales polimorfic: un obiect a cărui singură treabă e să construiască.

```
public interface IFabricaUtilizator
{
    string Tip { get; }
    Utilizator Creeaza(string[] campuri);
}
```

`FabricaStudent` știe să nască doar `Student`. `FabricaProfesor`, doar `Profesor`. Fiecare are `new`-ul ei, în clasa ei.

Iar cel care încarcă fișierul nu mai are niciun `switch` și niciun `new` de model:

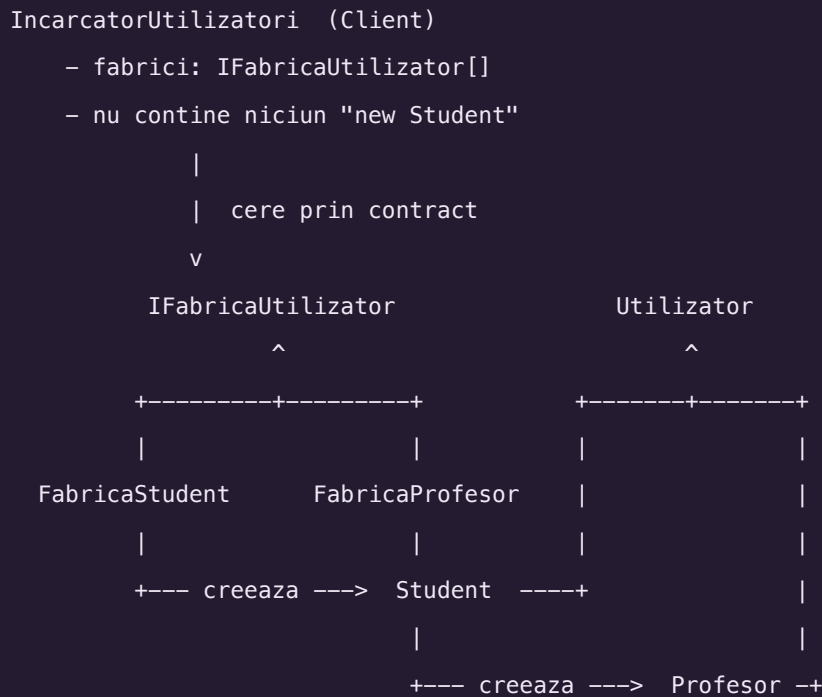
```
foreach (IFabricaUtilizator fabrica in fabrici)
{
    if (fabrica.Tip == campuri[0])
    {
        return fabrica.Creeaza(campuri);
    }
}
```

Observă ce e `if`-ul ăsta: o comparație între **două șiruri de date**, nu o verificare de tip.

`IncarcatorUtilizatori` nu cunoaște nicio clasă concretă de utilizator. Un tip nou de utilizator = **o clasă-model nouă + o fabrică nouă**, și zero linii modificate în încărcător.

Cele trei roluri

Rol	În exercițiu	Ce face
Product (contractul)	<code>Utilizator</code>	ce se construiește, văzut abstract
ConcreteProduct	<code>Student</code> , <code>Profesor</code>	produsele reale
Creator (contractul)	<code>IFabricaUtilizator</code>	declară metoda care naște un produs
ConcreteCreator	<code>FabricaStudent</code> , <code>FabricaProfesor</code>	fiecare, un singur <code>new</code>
Client	<code>IncarcatorUtilizatori</code>	cere un produs, fără să știe ce clasă primește



Citește diagrama pe orizontală: fiecare fabrică e legată de exact un produs. Asta e toată ideea — perechea (cine creează, ce creează) e închisă într-o clasă.

Mecanismul de dedesubt

Trei lucruri, toate cunoscute:

1. **new e o decizie luată la scriere, nu la rulare.** De-aia nu poate fi făcută polimorfic direct. Singura soluție e să o *muți* într-un obiect care poate fi ales la rulare.
2. **Indirecție.** Nu chemi constructorul, chemi pe cineva care îl cheamă pentru tine. Exact ce făcea **Comanda** cu strategia — doar că acolo delegai un *calcul*, aici delegi o *naștere*.
3. **Contractul ca tip de retur.** **Creeaza** întoarce **Utilizator**, nu **Student**. Clientul primește ceva despre care știe doar contractul. Dacă ar avea nevoie să știe tipul concret ca să-l folosească, n-ai câștigat nimic — de-aia **Utilizator** trebuie să aibă o metodă abstractă (**Descriere()**) care face treaba polimorfic.

Punctul 3 e cel pe care îl ratează majoritatea: o fabrică urmată de un **as** în client e o fabrică degeaba. Ai mutat **switch**-ul, nu l-ai eliminat.

Factory Method vs Simple Factory

Merită să știi că numele se folosesc amestecat, ca să nu te încurci când citești pe net.

- Ce faci în `ex10` — un contract de fabrică, cu mai multe implementări, alese la rulare — e forma pe care o vei folosi în 90% din cazuri. Unii o numesc **Simple Factory** sau *parameterized factory*.
- **Factory Method** în forma din carte (GoF) e mai strâmtă: o clasă de bază face o treabă mai mare și lasă o metodă abstractă prin care **subclasele decid produsul**. Adică fabrica nu e un obiect separat, e o metodă suprascrisă în ierarhia care oricum exista.

Ideea e aceeași în ambele: **muți `new` -ul în spatele unui contract**. Diferă doar unde stă metoda. Nu te bloca pe nume — recunoaște durerea și forma soluției.

Când îl folosești

- Trebuie să **alegi clasa la rulare**, după o valoare (un tip din fișier, o setare, o comandă a utilizatorului).
- Ai un `switch` /lanț de `if` care se termină în `new` — mirosul clasic.
- Vrei să poți **adăuga un tip nou** fără să atingi codul care le folosește.
- Construirea are pași sau validări proprii pe care nu vrei să le împrăștii prin cod.

Când NU îl folosești

- Ai **o singură clasă** de construit. `new Student(...)` direct e mai onest decât o fabrică cu o implementare.
- Construcția e banală și tipul e cunoscut la compilare. O fabrică acolo e ceremonie pură.
- Ai nevoie să **configurezi** un obiect cu multe câmpuri opționale — aia nu e problema de „ce clasă”, ci de „cum o umplu”. Pentru asta există **Builder**.

Capcane frecvente

- **Fabrică urmată de `as` în client**. Dacă după `Creeaza` faci `var s = rezultat as Student;`, ai mutat problema. Produsul trebuie folosit prin contract.
- **`switch` mutat în fabrică**. O singură clasă `FabricaUtilizatori` cu un `switch` înăuntru e mai ordonată decât înainte, dar tot trebuie deschisă la fiecare tip nou. Câte tipuri, atâtea fabrici.
- **Fabrica cu stare**. Ca la strategii: dacă fabrica reține ceva între apeluri, n-o mai poți refolosi liniștit.
- **`new` uitat în client**. Caută cuvântul `new` în clientul tău. Dacă apare vreun model concret acolo, pattern-ul nu e complet.

Legătura cu ce urmează

Factory Method răspunde la „**ce** clasă construiesc”. Rămâne întrebarea vecină: „**cum** o umplu, când are opt câmpuri, jumătate opționale, și doi vecini de constructor de același tip pe care îi poți inversa fără ca nimeni

să observe?" Uită-te la constructoarele `Teacher` și `Admin` din `academy`, unul sub altul, și la ordinea lui `salary` și `age`. Acolo va intra **Builder**.

Dar înainte de el vine ceva ce ai scris deja de trei ori, fără să știi cum se cheamă. Lecția 4.

Lecția 4 — Decorator

Un cuvânt: comportament adăugat prin împachetare. O clasă care semnează contractul **și ține înăuntru un alt obiect cu același contract**, îl cheamă, și îi ajustează rezultatul. Se pot stivui.

Nu e o lecție nouă. E un nume pentru ceva ce ai scris deja

Deschide trei fișiere din repo-ul tău, unul sub altul:

Fișier	Semnează	Ține înăuntru	Ce adaugă
ex1/Models/LivrareCuReducere.cs	ILivrareStrategie	ILivrareStrategie	scade un procent
ex6/Models/ComisionCuPlafon.cs	IComision	IComision	taie la un maxim
ex6/Models/ComisionCuBonus.cs	IComision	IComision	adaugă o sumă fixă

Toate trei au **aceeași formă**: implementează un contract și, în același timp, țin o instanță a aceluiași contract, pe care o cheamă și al cărei rezultat îl transformă.

```
public decimal Calculeaza(decimal valoareVanzare)
{
    return strategie.Calculeaza(valoareVanzare) + Bonus;
}
```

Asta e un **Decorator**, complet și funcțional. L-ai scris singur, la S3 din caiet, care era marcat **[provocare]** — iar întrebarea de la finalul exercițiului îți spunea deja că îi vom pune numele la lecția asta.

De ce merită un nume separat de Strategy

Structural seamănă: contract, mai multe clase, contextul nu știe ce ține. Diferența e **cine e înăuntru**.

	Strategy	Decorator
Ce ține clasa	contextul ține strategia	decoratorul ține un alt obiect de același tip cu el
Câte niveluri	unul	oricâte, stivuite
Ce face cu ce ține	îl întreabă, atât	îl întreabă și îi modifică răspunsul
Cine îl vede	contextul știe că are o strategie	apelantul nu știe că e împachetat

Ultima linie e miezul. În `Testare6` ai scris:

```
ComisionCuPlafon plafon = new(200.00m, new ComisionProcent(20));
ComisionCuBonus bonus = new(50.00m, plafon);
vanzare.SchimbaComision(bonus);
```

`Vanzare` primește un `IComision` și îl întreabă „cât e comisionul?”. Nu are idee că în spate sunt **trei** obiecte legate în lanț. Fiecare decorator e transparent pentru cel din afară — și exact de-aia se pot stivui oricât.

Cele trei roluri

Rol	În exercițiu	Ce face
Component (contractul)	<code>IComision</code>	operația pe care o adaugă și cel decorat, și decoratorul
ConcreteComponent	<code>ComisionProcent</code> , <code>ComisionFix</code>	fac treaba de bază, nu împachetează pe nimeni
Decorator	<code>ComisionCuPlafon</code> , <code>ComisionCuBonus</code>	semnează contractul, ține un <code>IComision</code> , îi ajustează rezultatul

```

Vanzare (apelantul)
|
| vede doar IComision
v
ComisionCuBonus (decorator) ---> +50
|
v
ComisionCuPlafon (decorator) ---> min(x, 200)
|
v
ComisionProcent (component) ---> 20% din 1500 = 300

```

Citește-l de jos în sus: `300` → `min(300, 200) = 200` → `200 + 50 = 250` . Exact ce scrie în output.

Ce ți-a lipsit din toate trei

Fiecare decorator al tău **își pierde identitatea a ce împachetează**:

```
public string Nume { get; } = "Cu bonus";
```

De-aia în output vezi `Cu bonus: 250.00` și nu poți spune ce e dedesubt. Un decorator adevărat compune și descrierea, nu doar calculul:

```
public string Nume => strategie.Nume + " + bonus";
```

Cu schimbarea asta, aceeași rulare scrie `Procent + plafon + bonus: 250.00` — și lanțul devine vizibil fără să te uiți în cod. **Trei linii, în trei fișiere**. Asta e toată tema lecției.

Mecanismul de dedesubt

1. **Aceeași interfață în două roluri.** Decoratorul e în același timp *implementare* a contractului și *client* al lui. De-aia poate sta oriunde stă un obiect obișnuit — inclusiv înăuntrul altui decorator.
2. **Compunere la runtime, nu la compilare.** Cu moștenire ai avea nevoie de o clasă pentru fiecare combinație: `ComisionProcentCuPlafon` , `ComisionProcentCuPlafonSiBonus` ... Cu decoratori, combinațiile se fac din obiecte, în momentul construirii. Trei decoratori dau opt combinații fără nicio clasă în plus.

3. **Ordinea contează.** `Bonus(50, Plafon(200, x))` dă `250`. `Plafon(200, Bonus(50, x))` dă `200`.

Aceleași piese, alt rezultat — pentru că fiecare decorator vede doar ce iese din cel de sub el.

Punctul 3 e cel care se uită. Un lanț de decoratori nu e o mulțime, e o **ordine**.

Când îl folosești

- Vrei să **adaugi** ceva peste un comportament existent, fără să atingi clasa care îl face.
- Ai nevoie de **combinații** de adăugiri, nu de o listă fixă.
- Apelantul nu trebuie să afle că s-a schimbat ceva — semnătura rămâne aceeași.
- Adăugirea e o **preocupare transversală**: jurnalizare, cache, reîncercare, limitare, validare.

Când NU îl folosești

- Ai o **singură** adăugire și nu se întrevăd altele: pune-o în clasa de bază și mergi mai departe.
- Adăugirea are nevoie să știe **ce anume** decorează (`if (component is ComisionFix)`) — atunci nu e decorator, e altceva.
- Lanțul crește peste 3–4 niveluri: depanarea devine grea, fiindcă stack trace-ul arată zece cadre pentru un singur apel.
- Ai nevoie ca **mai mulți** să reacționeze independent la același eveniment — aia e Observer. Decorator schimbă ce face operația; Observer *anunță* că s-a întâmplat ceva.

Capcane frecvente

- **Decorator care nu deleagă.** Dacă înăuntru nu apare niciun apel către obiectul împachetat, ai scris o strategie obișnuită cu un câmp nefolosit.
- **Identitate pierdută.** Exact ce ai acum: `Nume` hardcodat. Compune-l, altfel lanțul e invizibil.
- **Câmpul nevalidat.** `ComisionCuPlafon` verifică `comisionMax < 0`, `ComisionCuBonus` nu verifică nimic. Clase-frați scrise în aceeași ședință, una divergează — tiparul tău recurent.
- **Ordine presupusă.** Dacă rezultatul depinde de ordine (și depinde), scrie într-un test ambele variante, nu doar pe cea care-ți iese bine.

Legătura cu ce urmează

Decorator adaugă comportament **păstrând** contractul. Rămâne cazul vecin: ai un obiect care face exact ce-ți trebuie, dar **cu altă semnătură** — o bibliotecă străină, un format nou. Acolo intră **Adapter**, care nu adaugă nimic, doar traduce.

Iar în `academy`, `ITextMapper<T>` e locul unde le vei vedea pe amândouă: un decorator care validează linia înainte s-o dea mai departe, și un adaptor care pune JSON în spatele aceleiași interfețe.

Document viu — crește cu fiecare lecție. Rămase: State, Adapter, Builder, Singleton.

Document viu. Lecțiile de mai sus sunt cele scrise până acum. Urmează **State, Adapter, Builder** și **Singleton** — pagina se completează pe măsură ce sunt predate.



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro